

ESTUDO TÉCNICO • 10 MIN + DEMO

Testando um app de IA com ajuda de IA — JMeter para teste de carga

Da geração do plano ao diagnóstico automatizado — com alvo real, local e reproduzível.

Open WebUI 0.9.2 • Docker • <http://localhost:8080>

Segurança e Testes de Software - Guilherme de Oliveira Portas

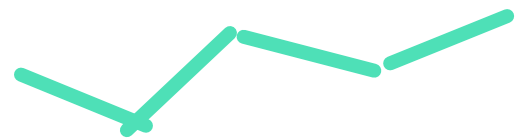


JMeter: protocolos, não navegador

Aplicativo Java open source que exercita protocolos e mede comportamento sob diferentes perfis de demanda.

CARGA

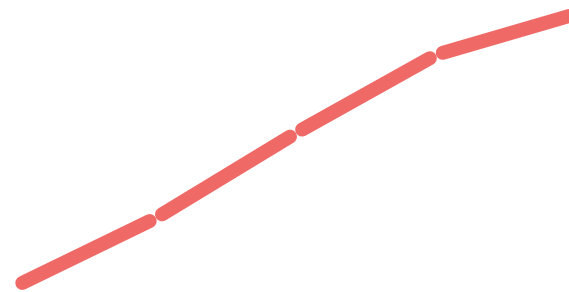
Demanda típica



Usuários simultâneos em operação normal.

STRESS

Até o limite



Capacidade máxima antes da falha.

SPIKE

Pico súbito



Reação a aumentos abruptos.

SOAK

Longa duração



Estabilidade sob carga prolongada.

GUI cria. CLI executa.

Carga real nunca roda na interface gráfica: ela consome recursos e não escala.

```
jmeter -n -t teste.jmx -l result.jtl -e -o ./report -f
```

-n

headless

-t

plano .jmx

-l

resultados .jtl

-e -o

relatório HTML

-f

sobrescrever saída

Cenário: Open WebUI 0.9.2

ALVO	Docker • http://localhost:8080
CARGA	50 threads • ramp-up 50 s • 10 loops
ENDPOINTS	GET /, /health, /api/version, /api/config, /manifest.json /api/v1/models, /api/v1/chats/list, /api/v1/prompts/list, /ollama/api/tags
AUTENTICAÇÃO	JWT extraído de POST /api/v1/auths/signin
SLAs	1 s health/version • 1,5–2 s APIs • 3 s home
FORA DO ESCOPO	/chat/completions (LLM)

Automatizar o repetível. Preservar o julgamento.

Testes manuais são lentos, repetitivos e sujeitos a erro. A IA encurta o ciclo; o engenheiro continua responsável pela validação e pela leitura do contexto.

01

GERA

plano parametrizado

02

OPERA

smoke, carga, stress

03

AJUDA

no diagnóstico

“A IA gera o plano, opera a execução e ajuda no diagnóstico — o engenheiro valida os achados.”

Framework de integração com IA

01

GERAR

Lê OpenAPI e controllers. Gera .jmx parametrizado; sugere JSON Extractor e CSV Data Set.

02

EXECUTAR

Opera smoke → carga → stress via CLI. Descobre e respeita ramp-up e rate-limit

03

DIAGNOSTICAR

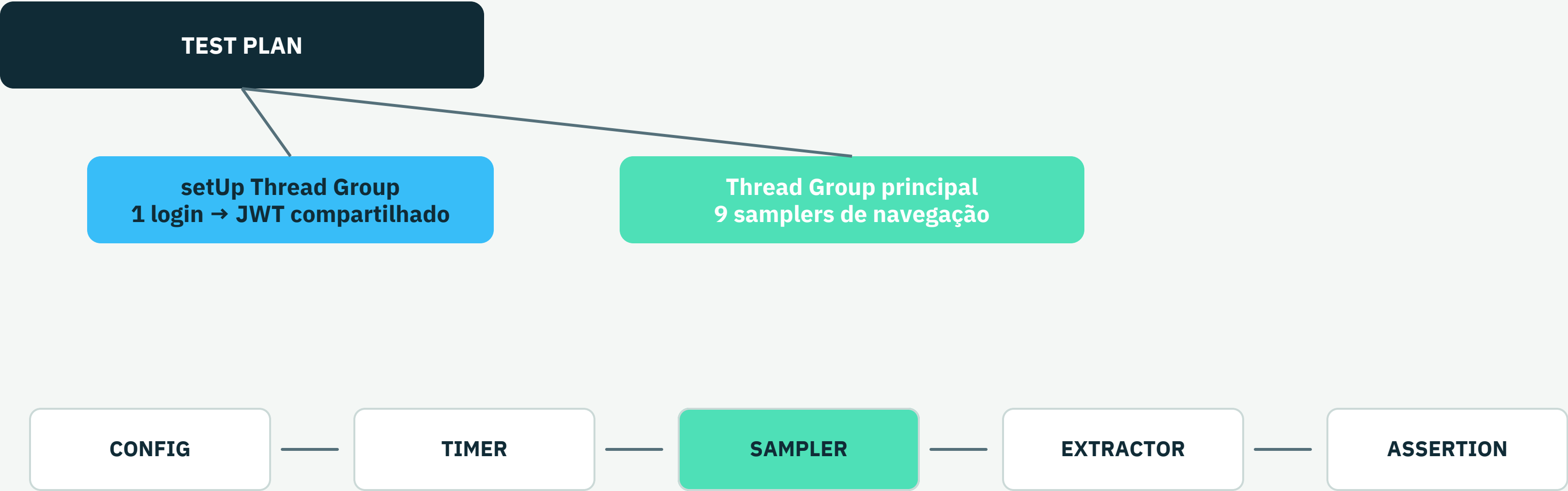
Parseia .jtl: p95, erro e throughput. Cruza com logs para separar script de sistema.

ALERTA

A IA alucina endpoint e cria assertion estrita demais — o smoke valida antes da carga.

Anatomia do Plano de Teste

Escopo desce na árvore. A execução segue uma ordem previsível.



Um plano. Várias escalas.

`${__P(threads,50)}`

A API de properties permite que o agente ajuste carga, host, porta, ramp-up e loops sem alterar o arquivo .jmx.

2

threads
SMOKE

50

threads
CARGA

```
jmeter -n -t teste.jmx -l result.jtl  
-Jhost=localhost -Jport=8080 -Jthreads=50 -Jramp=50 -Jloops=10
```

Agora: smoke ao vivo

Vou rodar uma validação rápida de ~10 segundos e, em seguida, abrir o dashboard da carga de 50 threads.

```
$ ./run.sh smoke
```

depois → [report-carga/](#)

O dashboard traz gráficos e métricas para análise detalhada do ambiente alvo.

O que quebrou na primeira versão?

ERRO	CAUSA	CORREÇÃO
429 Too Many Requests	Rate-limit com bloqueio de ~60 s	1 login no Setup; token compartilhado
401 em cascata	Token vazio após login falho	Setup garante token válido
304 Not Modified	Cache revalidando a SPA	Assertion aceita 200 ou 304

“Todo erro era um aprendizado, não lixo.”

Resultado final

4.501

amostras

0,87%

erro total

74 ms

média

≈ 42

req/s

39 erros restantes

Estouros de SLA de ~4,5 s no pico do ramp-up. O app single-process sofre contenção no burst; a leitura correta usa p95 por endpoint, não apenas a média.

P95 > MÉDIA

Fluxo seguro, leitura responsável

1 Montar plano e parametrizar o cenário

2 Validar rapidamente com smoke

3 Executar a carga completa

4 Ler p95 e taxa de erro

5 Iterar: corrigir e ajustar

Teste de carga também é teste de resiliência — o rate-limit 429 que nos bloqueou é um controle de segurança que o teste revelou.